CNG 514
Assignment 3: Association Mining and Clustering

Student Name: Furkan Duman
Student Number: 2453173
Date of Submission: 22 May 2024

Contents

1 Identify and characterize a dataset **3**
1.1 Overview of the Dataset 3
1.2 General Structure and Explanations of the Code 3
1.3 Analysis of Frequent Itemsets and the Impact of Changing Support and Confidence Values 5

2 Part 2: Clustering **9**
2.1 Task Overview 9
2.2 General Structure and Explanations of the Code 9
2.3 Summary 13

3 References **13**

List of Figures

1 Previous studies 3
2 Importing Necessary Libraries 3
3 Reading and Preparing the Data 4
4 Transforming Data Using TransactionEncoder 4
5 Finding Frequent Itemsets Using Apriori Algorithm 4
6 Finding Itemsets with Maximum Length 5
7 Generating Association Rules 5
8 Rules with their antecedents and consequents 6
9 Commonly Looked Segments 7
10 Effect Of Increasing Support Value 8
11 Importing Libraries 9
12 Defining Data Directory And Listing Participant Files 10
13 Processing Each File 10
14 Data Processing- Warning 10
15 Concatenating Data 11
16 Applying DBSCAN 11
17 Removing Noise Points 11
18 Loading Web Page Image And Plotting Clusters on Web Page Image 12
19 Clusters 13

1 Identify and characterize a dataset

1.1 Overview of the Dataset

The dataset provided in this assignment pertains to eye-tracking data, which records the fixations of users - specifically, the points on a webpage where the users looked at and the duration of their gaze. Eye-tracking is a powerful technique used to understand user attention and behavior on a given stimulus, such as a webpage.

The dataset consists of sequences from 36 people, each indicating the segments they looked at on the Apple homepage. Each line in the dataset represents a user's sequence, totaling 37 lines, including the header or additional metadata.

Each user's sequence is a list of pairs, where each pair contains:

- A segment label (e.g., 'I', 'E', 'C', etc.), representing different parts of the Apple homepage.
- The duration in milliseconds (ms) that the user spent looking at that segment.

The main objective of this part of the assignment is to process the sequences from the given dataset and identify the frequent itemsets of webpage segments using the Apriori algorithm. These frequent itemsets will indicate the most popular segments and the co-occurrences of these segments that people look at.

In previous studies[[1]], Trending Scanpaths were found for the Apple page using this eye data and the STA algorithm. My aim in this assignment is to find Association Rules using Apriori.

Page Name	Expected Sequence	#	Trending Scanpath
Apple	EB	34	I C F H **E** B G **B**

Figure 1: Previous studies

1.2 General Structure and Explanations of the Code

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
import matplotlib.pyplot as plt
```

Figure 2: Importing Necessary Libraries

- **pandas** : Used for data processing and analysis.

- **matplotlib.pyplot**: Used for plotting graphs.
- **mlxtend.frequent_patterns**: Used for the Apriori algorithm and association rules.
- **mlxtend.preprocessing:** Used for TransactionEncoder to transform data.

```
data = []
with open("Apple_sequence_dataset.txt", "r") as file:
    for line in file:
        segments = [segment.strip().split(",")[0] for segment in line.strip()[2:-2].split("], ")]
        data.append(segments)
```

Figure 3: Reading and Preparing the Data

- An empty list called **data** is created.
- The file **Apple_sequence_dataset.txt** is read.
- Segments in each line are extracted, cleaned of unnecessary characters, and added to the data list. Here, **strip** and **split** methods are used to clean and parse the segments.

```
te = TransactionEncoder()
te_ary = te.fit(data).transform(data)
df = pd.DataFrame(te_ary, columns=te.columns_)
```

Figure 4: Transforming Data Using TransactionEncoder

- A TransactionEncoder object is created.
- Using fit and transform methods, the data list is converted into a binary format. This format indicates the presence (1) or absence (0) of each segment.
- The transformed data is transferred to a DataFrame, and the column names are set to the segment names.

```
frequent_itemsets = apriori(df, min_support=0.75, use_colnames=True)
```

Figure 5: Finding Frequent Itemsets Using Apriori Algorithm

- The apriori function is used with a minimum support value of 75%. This step finds frequent itemsets that appear in at least 75% of the users.
- **The use_colnames**=True parameter ensures that segment names are used in the results.

The reason for the maximum length is association rules are derived from frequent itemsets. Knowing the maximum length helps to comprehensively understand the dependencies among items. Additionally, by determining the maximum length, we can optimize performance and reduce computational overhead.

```
max_length = frequent_itemsets['itemsets'].apply(lambda x: len(x)) == max(frequent_itemsets['itemsets'].apply(len))
```

Figure 6: Finding Itemsets with Maximum Length

- The lengths of the obtained frequent itemsets are calculated.
- The **apply** function is used to find the length of each itemset.
- The itemsets with the maximum length are determined.

```
rules = association_rules(frequent_itemsets[max_length], metric="confidence", min_threshold=0.75, support_only=True)

print("\nAssociation Rules with Confidence:")
print(rules[['antecedents', 'consequents']])
```

Figure 7: Generating Association Rules

- The association_rules function is used to create association rules with a minimum confidence value of 75
- The metric="confidence" parameter specifies that the confidence metric is used to generate the rules. The min_threshold=0.75 parameter sets the minimum confidence value to 75

1.3 Analysis of Frequent Itemsets and the Impact of Changing Support and Confidence Values

My output consists of 510 association rules derived from the dataset. Here's a sample of the rules with their antecedents and consequents:

```
Association Rules with Confidence:
                                antecedents                                consequents
0    ('J', 'C', 'E', 'B', 'D', 'G', 'F', 'H')                                   ('I')
1    ('J', 'C', 'E', 'B', 'D', 'G', 'F', 'I')                                   ('H')
2    ('J', 'C', 'E', 'B', 'D', 'G', 'H', 'I')                                   ('F')
3    ('J', 'C', 'E', 'B', 'D', 'F', 'H', 'I')                                   ('G')
4    ('J', 'C', 'E', 'B', 'G', 'F', 'H', 'I')                                   ('D')
..                                       ...                                     ...
505                                    ('D')  ('J', 'C', 'E', 'B', 'G', 'F', 'H', 'I')
506                                    ('G')  ('J', 'C', 'E', 'B', 'D', 'F', 'H', 'I')
507                                    ('F')  ('J', 'C', 'E', 'B', 'D', 'G', 'H', 'I')
508                                    ('H')  ('J', 'C', 'E', 'B', 'D', 'G', 'F', 'I')
509                                    ('I')  ('J', 'C', 'E', 'B', 'D', 'G', 'F', 'H')

[510 rows x 2 columns]
```

Figure 8: Rules with their antecedents and consequents

The frequent itemsets in my output are quite long, involving combinations of up to nine different segments. This suggests complex patterns of segment co-occurrence, meaning users navigate through multiple related segments rather than looking at isolated ones. Segments 'J', 'C', 'E', 'B', 'D', 'G', 'F', 'H', and 'I' often appear together in various combinations.

Figure 9: Commonly Looked Segments

The frequent itemsets in my output seem to be quite long, involving combinations of up to nine different segments This implies that there are complex patterns of segment co-occurrence, indicating that users do not just look at isolated segments but tend to navigate through multiple related segments The segments 'J', 'C', 'E', 'B', 'D', 'G', 'F', 'H', and 'I' appear frequently in various combinations.

Impact of Changing Support and Confidence Values

1. **Increasing Support Value:**

   - If I increase the support value from 75% to, say, 95%, the number of

```
Association Rules with Confidence:
                  antecedents                consequents
0   ('I', 'G', 'F', 'C', 'E')                      ('H')
1   ('I', 'G', 'F', 'C', 'H')                      ('E')
2   ('I', 'G', 'F', 'E', 'H')                      ('C')
3   ('I', 'G', 'C', 'E', 'H')                      ('F')
4   ('I', 'F', 'C', 'E', 'H')                      ('G')
..                        ...                        ...
57                      ('G')  ('I', 'F', 'C', 'E', 'H')
58                      ('F')  ('I', 'G', 'C', 'E', 'H')
59                      ('C')  ('I', 'G', 'F', 'E', 'H')
60                      ('E')  ('I', 'G', 'F', 'C', 'H')
61                      ('H')  ('I', 'G', 'F', 'C', 'E')

[62 rows x 2 columns]
```
[H]

Figure 10: Effect Of Increasing Support Value

frequent itemsets will likely decrease.

- This will filter out itemsets that are less common, focusing only on those that a large majority of users look at.

2. **Decreasing Support Value:**
   - If I decrease the support value to 50%, more itemsets will be identified as frequent.
   - This broader view will include less common patterns, providing a more comprehensive understanding of user behavior. However, it might also introduce noise by including segments that are not as significantly viewed by the majority.

3. **Increasing Confidence:**
   - Raising the confidence threshold (e.g., from 75% to 90%) results in stricter rules, meaning the rules will only include the most reliable segment associations. This can improve the precision of the rules but may lead to fewer rules being generated.

4. **Decreasing Confidence:** It allows for more associations to be identified but can reduce the reliability of the rules, including associations that are less strongly supported by the data.

2 Part 2: Clustering

2.1 Task Overview

The task involves eye-tracking data collected from 38 participants viewing the Apple home page. The dataset contains the following features for each fixation:

- **FixationIndex**: Unique index for each fixation.
- **Timestamp**: Incremental timestamp of the fixation.
- **FixationDuration**: Duration of the fixation.
- **MappedFixationPointX**: X-coordinate of the fixation point.
- **MappedFixationPointY**: Y-coordinate of the fixationT point.
- **StimuliName**: URL of the web page.

Participants were given specific tasks. While they were performing these tasks, their eye data was kept and which segments they looked at were recorded.

The objective is to use the DBSCAN clustering algorithm to identify and visualize clusters of fixation points on the web page image (Apple.png).

2.2 General Structure and Explanations of the Code

```
import os
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from sklearn.cluster import DBSCAN
```

Figure 11: Importing Libraries

- **os** is used for file and directory operations.
- **pandas** is used for data processing and analysis.
- **matplotlib.pyplot** is used for plotting, and matplotlib.image is used for image processing.

- **DBSCAN** is imported from the sklearn library to apply the density-based clustering algorithm.

```
data_dir = 'Apple_fixation_dataset'
data_files = [f for f in os.listdir(data_dir) if f.startswith('P-') and f.endswith('.txt')]
```

Figure 12: Defining Data Directory And Listing Participant Files

- The directory where the data files are located is defined. This line lists all participant files in the specified directory that start with P- and end with .txt.

```
all_data = []
for file in data_files:
    file_path = os.path.join(data_dir, file)
    # Check if the file is not empty
    if os.path.getsize(file_path) > 0:
        try:

            df = pd.read_csv(file_path, delimiter='\t')
            required_columns = ['FixationIndex', 'Timestamp', 'FixationDuration', 'MappedFixationPointX', 'MappedFixationPointY'
            if not df.empty and all(column in df.columns for column in required_columns):
                df['Participant'] = file  # Add a column for participant identifier
                all_data.append(df)

            else:
                print(f"Warning: {file} does not contain the required columns or is empty and will be skipped.")
        except pd.errors.ParserError:
            print(f"Warning: {file} is not in the expected format and will be skipped.")
        except pd.errors.EmptyDataError:
            print(f"Warning: {file} is empty or not in the expected format and will be skipped.")
    else:
        print(f"Warning: {file} is empty and will be skipped.")
```

Figure 13: Processing Each File

- This loop processes each file by:
- Constructing the file path.
- Checking if the file is non-empty.

  During the data preprocessing stage, the following warnings were encountered and handled:

```
Warning: P-38.txt is empty and will be skipped.
```

Figure 14: Data Processing- Warning

- If the file is non-empty, reading it into a DataFrame and validating if the DataFrame contains the required columns and is non-empty.
- Adding a Participant column to identify the participant and appending the DataFrame to the **all_data** list if valid.
- Printing a warning if the file does not contain the required columns, is empty, or is in an incorrect format.

```
if all_data:
    all_data_df = pd.concat(all_data, ignore_index=True)
```

Figure 15: Concatenating Data

If there is any valid data, all individual DataFrames are concatenated into a single DataFrame.

```
# Initialize DBSCAN
dbscan = DBSCAN(eps=30, min_samples=10)

# Fit DBSCAN to the data
clusters = dbscan.fit_predict(eye_tracking_data[['MappedFixationPointX', 'MappedFixationPointY']])

# Remove noise points (-1 cluster)
cleaned_data = eye_tracking_data[clusters != -1]
```

Figure 16: Applying DBSCAN

In this part, I applied the DBSCAN clustering algorithm to the X and Y coordinates of the fixation points, using **eps=30** and **min_samples**=10. **The eps value of 30** was chosen to define the maximum distance between two points for them to be considered as in the same neighborhood, ensuring that only closely located fixation points form a cluster. **The min_samples value of 10** was selected to set the minimum number of points required to form a dense region, ensuring that each cluster represents a significant area of user interest.

```
cleaned_data = eye_tracking_data[clusters != -1]
```

Figure 17: Removing Noise Points

This line **removes noise** points identified by DBSCAN (those assigned a cluster label of -1).

```
web_page_image = mpimg.imread('Apple.png')
plt.figure(figsize=(12, 8))
plt.imshow(web_page_image)
plt.scatter(cleaned_data['MappedFixationPointX'], cleaned_data['MappedFixationPointY'], c=clusters[clusters != -1],
            cmap='viridis', alpha=0.5)
plt.title('Eye Tracking Clusters on Apple Web Page')
plt.xlabel('X Coordinate')
plt.ylabel('Y Coordinate')
plt.show()
```

Figure 18: Loading Web Page Image And Plotting Clusters on Web Page Image

This section plots the cluster data on the web page image:

- Loads the image as the background. Plots the cleaned fixation points, coloring them by cluster.
- Sets the title and labels for the plot. Displays the plot.

Figure 19: Clusters

The clusters formed in the figure matched the expected segments. The segments from the frequent item sets created in the first part are almost identical to these clusters. These segments represent the areas of common interest among users. Various studies can be conducted using these segments.

2.3 Summary

The code successfully completed the steps of loading, cleaning, summarizing, clustering, and visualizing the eye-tracking data, allowing us to identify the primary areas of interest on the web page. This analysis provides valuable insights for enhancing web page design and user experience.

3 References

1. Eraslan, Sukru Yesilada, Yeliz Harper, Simon. (2016). Scanpath Trend Analysis on Web Pages: Clustering Eye Tracking Scanpaths. ACM Transactions on the Web. 10. 35 pages. 10.1145/2970818.